FIAP
PÓS-TECH

FIAP · Pós-Tech

TECH CHALLENGE — FASE 4

Arquitetura e Desenvolvimento em Java

Plataforma de Feedback Serverless

Documentação técnica do Feedback-Serverless. Reúne o recebimento das avaliações das aulas, o aviso imediato aos administradores quando uma avaliação é crítica e o relatório semanal de satisfação, em três funções serverless na Azure sobre um Azure SQL serverless e uma Storage Queue.

DISCIPLINA

Arquitetura e Desenvolvimento em Java

TURMA E GRUPO

Turma 12ADJT · Grupo 46

INTEGRANTES

Gustavo Chauar Correa (RM 370978)
Davi Faustino Souza Lima (RM 373019)
Caroline Da Silva Brunheti (RM 372744)
Eduardo Mendes Germano (RM 371760)
Roberto Domingos Locatelli (RM 370928)

REPOSITÓRIO

github.com/JavaPosTech/Feedback-Serverless

ENTREGA E VERSÃO

Novembro de 2026 · versão 1.0.0

Sumário

1. Introdução **3**

Descrição do problema · Objetivo do projeto 3

2. Arquitetura do Sistema **4**

As três funções · Camadas · Tecnologias · Decisões · Modelo de cloud · Topologia · Estrutura de pastas 4

3. ReceberAvaliacaoFunction: Função de Entrada **8**

Endpoint · Estrutura · Fluxo · Regra de urgência · Validações · Erros · Exemplos 8

4. NotificarUrgenciaFunction: Função de Notificação **12**

Estrutura · Fluxo · Contrato da mensagem · Resiliência · Exemplo 12

5. GerarRelatorioFunction: Função de Relatório **15**

Estrutura · Agendamento · Consultas · Banco de dados · Exemplo 15

6. Configuração e Execução **18**

Arquivos · Variáveis · Passo a passo · Deploy · Monitoramento · Segurança · URLs e portas 18

7. Qualidade do Código **23**

Boas práticas · SOLID e DRY · Testes · Integração contínua 23

8. Repositório do Código **24**

1. Introdução

Descrição do problema

Para garantir a qualidade dos cursos on-line, é essencial que os estudantes possam **avaliar as aulas** e que os administradores possam **acompanhar rapidamente a satisfação** dos alunos. O sistema deve receber os feedbacks, **avisar sobre os itens críticos** assim que eles chegam e gerar **relatórios periódicos** que ajudem na análise dos dados.

O desafio da fase pede que essa solução rode em **ambiente de nuvem**, com **funções serverless** de responsabilidade única. Como os créditos de cloud são limitados, nada fica ligado sem uso: cada função só executa quando o seu gatilho acontece, e o banco de dados pausa sozinho quando ninguém o consulta.

Objetivo do projeto

Desenvolver uma aplicação hospedada na **Azure**, com funções serverless que automatizam o recebimento dos feedbacks, o envio das notificações e a geração dos relatórios, com segurança dos dados, governança de acesso, monitoramento e infraestrutura como código. Concretamente, a fase entrega um repositório e um Function App com três funções complementares:

- **ReceberAvaliacaoFunction**: função HTTP que recebe o `POST /avaliacao`, valida a entrada, calcula a urgência pela nota, grava a avaliação no Azure SQL e, quando ela é **crítica**, publica uma mensagem na fila `avaliacoes-criticas`.
- **NotificarUrgenciaFunction**: função acionada pela fila, que envia aos administradores o **e-mail de urgência** com descrição, urgência e data de envio. Não acessa o banco: tudo o que precisa chega na mensagem.
- **GerarRelatorioFunction**: função agendada para **toda segunda-feira às 08:00** de Brasília, que calcula a média das notas, a quantidade de avaliações por dia e por urgência dos últimos 7 dias e envia o **relatório semanal**. Só lê.

> **Uma função, um gatilho, uma responsabilidade.** Cada função é uma classe com um único método anotado com `@FunctionName`, e a regra de negócio fica nos services. A entrada não envia e-mail, a notificação não lê o banco e o relatório não grava nada. Por isso uma falha no envio de e-mail nunca impede que uma avaliação seja gravada, e cada função pode falhar, escalar e ser reprocessada sem afetar as outras.

2. Arquitetura do Sistema

As três funções em um olhar

Função	Responsabilidade	Gatilho	Acesso
ReceberAvaliacao	Valida, classifica, grava e enfileira as avaliações críticas.	HTTP `POST /avaliacao`	Chave de função
NotificarUrgencia	Envia o e-mail de urgência aos administradores.	Fila `avaliacoes-criticas`	Interno
GerarRelatorio	Agrega a semana e envia o relatório com média e distribuições.	Timer `0 0 11 * * 1`	Interno

Descrição da arquitetura

O projeto segue uma arquitetura em camadas com direção de dependência única: cada camada conhece apenas a imediatamente abaixo. O código é organizado por camada e, dentro de cada camada, por assunto, então a estrutura de pacotes revela ao mesmo tempo a responsabilidade técnica e o assunto de negócio. A função é só a fronteira com o runtime do Azure Functions: recebe o gatilho, chama o service e devolve a resposta. Criar uma nova função significa replicar a mesma fatia (função → service → repository ou client → testes).

Camadas da aplicação

Camada	Responsabilidade
Function	Fronteira da aplicação. Recebe o gatilho (HTTP, fila ou timer), converte a entrada e a saída. Sem regra de negócio.
Service	Regra de negócio, validação (`@Valid`), classificação da urgência, agregação do relatório e transações (`QuarkusTransaction`).
Repository	Acesso a dados com Panache (`PanacheRepositoryBase`) e as consultas JPQL do relatório.
Client	Integração com a Azure pelos SDKs oficiais: Storage Queue (`FilaClient`) e Communication Services (`EmailClient`).

Camada	Responsabilidade
Model / Template	Entidade JPA, DTOs de entrada e saída (records imutáveis), mapper MapStruct e templates Qute dos e-mails.
Exceptions / Config	Exceções de negócio, tratamento centralizado de erros (`FunctionExceptionHandler`) e configuração tipada (`FeedbackConfig`, `AzureConfig`).

Tecnologias utilizadas

Item	Tecnologia	Versão
Linguagem	Java	21
Framework	Quarkus com a extensão `quarkus-azure-functions`	3.40.1
Build	Gradle (Kotlin DSL)	9.8.0
Serverless	Azure Functions, plano Flex Consumption (Linux)	runtime 4
Banco de dados	Azure SQL Database serverless · SQL Server 2022 no ambiente local	—
Migrações	Flyway	—
Persistência	Hibernate ORM com Panache	—
Fila	Azure Storage Queue · Azurite no ambiente local	Azurite 3.36.0
E-mail	Azure Communication Services (Email)	—
SDKs da Azure	azure-identity, azure-storage-queue, azure-communication-email	BOM 1.3.9
Templates	Qute, com `@CheckedTemplate`	—
Mapeamento	MapStruct	1.6.3
Boilerplate	Lombok	1.18.48
Segredos	Azure Key Vault	—
Monitoramento	Application Insights + Log Analytics	—

Item	Tecnologia	Versão
Infraestrutura	Bicep (infraestrutura como código)	—
Testes	JUnit 5 com `@QuarkusTest`	—
Ambiente local	Docker / Docker Compose	—
Integração contínua	GitHub Actions	—

Decisões de arquitetura

- **Fila entre a entrada e a notificação.** O estudante recebe a resposta na hora, uma falha no envio do e-mail não perde a avaliação e a Storage Queue tenta de novo sozinha: são 5 tentativas (`maxDequeueCount` no `host.json`) e depois a mensagem vai para `avaliacoes-criticas-poison`.
- **Azure SQL serverless.** O relatório precisa de média e de contagens por dia e por urgência, o que é natural em SQL. O banco `GP_S_Gen5` começa em 0,5 vCore, pausa após 60 minutos sem uso e usa a oferta gratuita (`useFreeLimit`), pausando quando ela acaba em vez de gerar cobrança.
- **Plano Flex Consumption.** É o plano serverless atual da Azure para Functions no Linux: cobrança por execução, escala até zero, suporte a Java 21 e até 40 instâncias de 2 GB.
- `quarkus-azure-functions`**, e não a variante** `-http`**.** A versão `-http` coloca uma API REST inteira dentro de uma única função, o que esconderia a separação em três funções de responsabilidade única.
- **Identidade gerenciada no lugar de senhas.** O Function App acessa o banco (`authentication=ActiveDirectoryMSI`), a fila e o Key Vault com a própria identidade, autorizada por papéis RBAC de menor privilégio.
- **Mensagem autossuficiente.** A `AvaliacaoCriticaDTO` leva id, descrição, nota, urgência e data de envio, então a função de notificação não depende do banco para montar o e-mail.
- **Datas no horário de Brasília.** As avaliações são gravadas em `America/Sao_Paulo` (`feedback.fuso-horario`), para que o relatório agrupe pelos dias que os administradores enxergam. O timer roda em UTC, por isso o cron `0 0 11 * * 1` corresponde às 08:00 de segunda.
- **Infraestrutura como código.** Todo o ambiente é descrito em Bicep (`main.bicep` e 7 módulos) e parametrizado por `dev.bicepparam`, então o ambiente pode ser recriado do zero com um único comando.

Modelo de cloud escolhido

Serviço

FaaS (serverless) para as funções, com PaaS gerenciado para banco, fila, e-mail e segredos. Não há servidor nem sistema operacional para administrar.

`Azure Functions · PaaS`

Implantação

Nuvem pública Azure, na região `brazilsouth`, a mesma plataforma usada no conteúdo da fase. Os dados de e-mail ficam no Brasil.

`Nuvem pública`

Custo

Funções e banco escalam até zero, o banco usa a oferta gratuita e um orçamento mensal avisa sobre gastos. Importante com créditos limitados.

`Pague pelo uso`

Topologia na Azure

Todos os recursos ficam em um único resource group por ambiente. O Function App hospeda as três funções e fala com os serviços de apoio sempre pela sua identidade gerenciada, sem senha no código ou nas configurações.

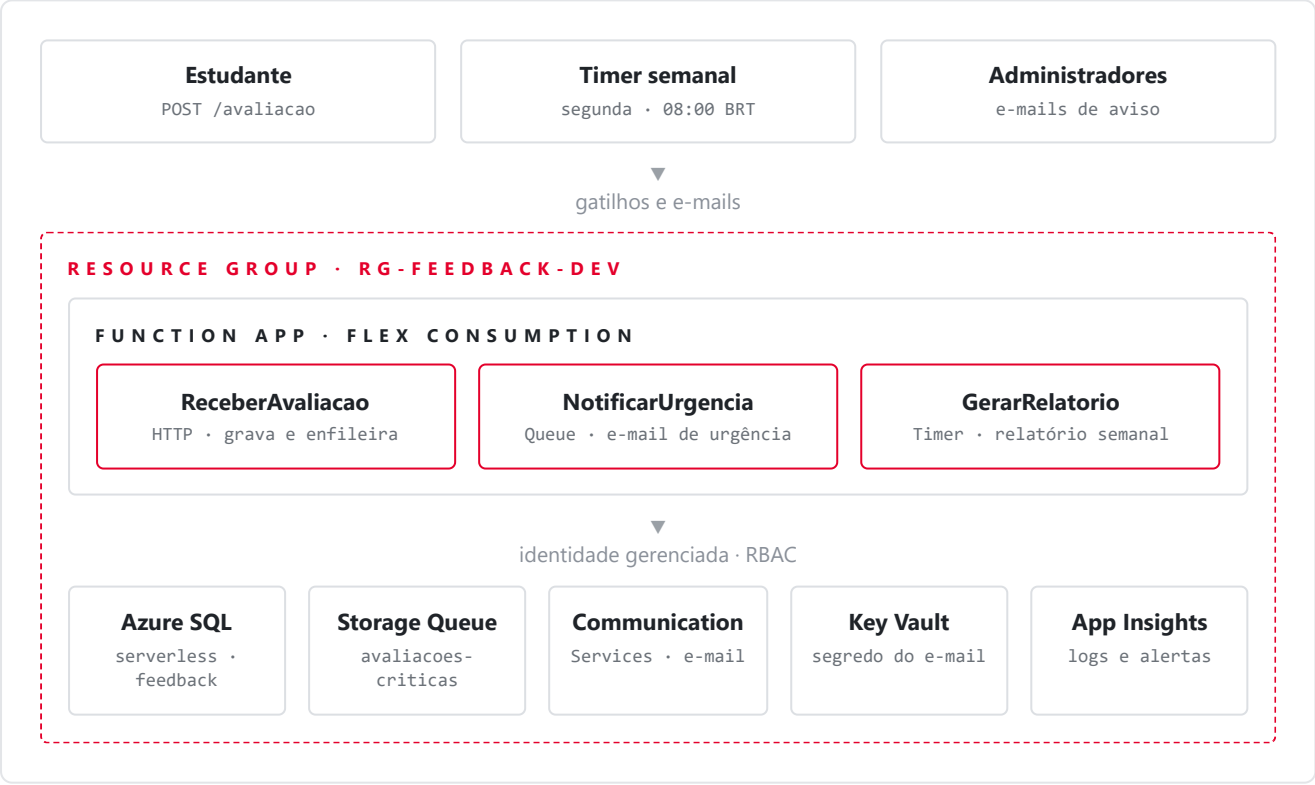


Figura 1. Recursos da Fase 4 na Azure

A `ReceberAvaliacao` grava no Azure SQL e publica na Storage Queue apenas as avaliações críticas. A `NotificarUrgencia` consome a fila e envia o e-mail pelo Communication Services. A `GerarRelatorio` lê o Azure SQL e envia o relatório pelo mesmo canal. O Key Vault guarda o único segredo da solução, e o Application Insights recebe os logs, as métricas e a execução de cada função.

Estrutura de pastas

```
src/main/java/br/com/fiap/feedback/
├── function/      # ReceberAvaliacao, NotificarUrgencia, GerarRelatorio
├── service/       # AvaliacaoService, NotificacaoService, RelatorioService
├── repository/    # AvaliacaoRepository: Panache + consultas do relatório
├── client/        # FilaClient (Storage Queue), EmailClient (Communication Services)
├── template/      # EmailTemplates (Qute) e FormatacaoExtensions
├── model/
│   ├── entity/    # Avaliacao (entidade JPA, Lombok)
│   ├── dto/       # avaliacao/, notificacao/ e relatorio/ (records)
│   └── mapper/    # AvaliacaoMapper (MapStruct)
├── enums/         # Urgencia e a regra deNota(int)
├── exceptions/    # exceções, dto/ e handler/FunctionExceptionHandler
└── config/        # FeedbackConfig, AzureConfig (clientes do SDK)
src/main/resources/
├── application.properties     # configuração por perfil
├── db/migration/              # V1__CriarTabelaAvaliacao.sql
└── templates/email/           # urgencia.html, relatorioSemanal.html
infra/                         # main.bicep, dev.bicepparam e modules/
```

3. ReceberAvaliacaoFunction: Função de Entrada

Função HTTP responsável pela entrada das avaliações. É a única função exposta para fora da Azure e é protegida por **chave de função** (`AuthorizationLevel.FUNCTION`). O `host.json` remove o prefixo padrão `/api`, então a rota publicada é exatamente `/avaliacao`.

Endpoint

Método	Rota	Autorização
`POST`	`/avaliacao`	Chave de função, no parâmetro `?code=<chave>` ou no cabeçalho `x-functions-key`

Estrutura

```
function/ReceberAvaliacaoFunction.java      # @HttpTrigger · POST /avaliacao
service/AvaliacaoService.java              # valida, classifica, grava e enfileira
repository/AvaliacaoRepository.java        # persist (Panache)
client/FilaClient.java                     # publica na Storage Queue
model/dto/avaliacao/                       # SalvarAvaliacaoDTO, AvaliacaoResponseDTO
model/mapper/AvaliacaoMapper.java          # conversões com MapStruct
enums/Urgencia.java                        # regra deNota(int)
exceptions/handler/FunctionExceptionHandler.java  # erros no formato único
```

Fluxo de entrada

1. A função lê o corpo da requisição e o converte em `SalvarAvaliacaoDTO` com o `ObjectMapper`.
2. `AvaliacaoService.salvar(...)` valida o DTO (`@Valid`), calcula a urgência com `Urgencia.deNota(...)`, registra a `dataEnvio` no horário de Brasília e grava a avaliação, dentro de uma transação.
3. Se a urgência for `CRITICA`, depois da transação confirmada, `FilaClient.publicar(...)` envia a `AvaliacaoCriticaDTO` para a fila `avaliacoes-criticas`.
4. A função responde `201 Created` com a avaliação gravada. Qualquer exceção é entregue ao `FunctionExceptionHandler`, que monta a resposta de erro.

Regra de urgência

O enunciado não define como classificar uma avaliação, então o grupo escolheu a regra abaixo. Ela fica em um único lugar, o método `Urgencia.deNota(int)`, e é a única regra que decide o fluxo: só as avaliações críticas geram aviso imediato.

Nota	Urgência	Efeito
0 a 3	`CRITICA`	E-mail imediato aos administradores, via fila
4 a 6	`MEDIA`	Entra só no relatório semanal
7 a 10	`BAIXA`	Entra só no relatório semanal

Validações

Campo	Regra	Mensagem
`nota`	Obrigatória, inteira, de 0 a 10	A nota é obrigatória! · A nota deve estar entre 0 e 10!
`descricao`	Obrigatória, não vazia, até 500 caracteres	A descrição é obrigatória! · A descrição deve ter no máximo 500 caracteres!

> **Validação em três níveis.** O Bean Validation recusa a requisição antes de qualquer gravação, o `Urgencia.deNota` recusa notas fora da faixa caso a regra seja chamada por outro caminho, e o banco reforça os limites com as constraints `ck_avaliacao_nota` e `ck_avaliacao_urgencia`.

Erros comuns

Código	Título	Quando ocorre
`400`	Requisição Inválida!	Um ou mais campos inválidos. A lista vem em `errors`, com `campo` e `mensagem`.
`400`	Corpo da Requisição Inválido!	Corpo ausente, JSON malformado ou com tipo errado.
`400`	Nota Inválida!	Nota fora de 0 a 10 recusada pela regra de urgência.
`401`	—	Chave de função ausente ou inválida, recusada pelo próprio runtime.
`500`	Erro Interno!	Falha inesperada. O detalhe fica só no log e no Application Insights.

Os erros da aplicação saem como `application/problem+json`, no formato `ErrorResponseDTO` com `status`, `title`, `instance`, `type`, `detail`, `errors` e `timestamp`, produzido pelo `FunctionExceptionHandler`.

Exemplos de requisição e resposta

1) Avaliação crítica

```
POST /avaliacao?code=<chave>
Content-Type: application/json

{
  "nota": 2,
  "descricao": "Não consegui acompanhar a aula, o áudio cortou várias vezes"
}
```

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": "3f1c2b7e-8a4d-4c3b-9e2f-1a2b3c4d5e6f",
  "nota": 2,
  "descricao": "Não consegui acompanhar a aula, o áudio cortou várias vezes",
  "urgencia": "CRITICA",
  "dataEnvio": "2026-10-12T19:45:00.512493"
}
```

2) Mensagem publicada na fila

```
// fila: avaliacoes-criticas · conteúdo codificado em Base64 pelo FilaClient
{
  "id": "3f1c2b7e-8a4d-4c3b-9e2f-1a2b3c4d5e6f",
  "descricao": "Não consegui acompanhar a aula, o áudio cortou várias vezes",
  "nota": 2,
  "urgencia": "CRITICA",
  "dataEnvio": "2026-10-12T19:45:00.512493"
}
```

3) Requisição inválida

```
POST /avaliacao?code=<chave>
Content-Type: application/json

{ "nota": 11, "descricao": "" }
```

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "status": 400,
  "title": "Requisição Inválida!",
  "instance": "/avaliacao",
  "type": "/Feedback-Serverless/problems/requisicao-invalida",
  "detail": "Um ou mais campos estão inválidos!",
  "errors": [
    { "campo": "nota", "mensagem": "A nota deve estar entre 0 e 10!" },
    { "campo": "descricao", "mensagem": "A descrição é obrigatória!" }
  ],
  "timestamp": "2026-10-12T19:46:10.120334"
}
```

4. NotificarUrgenciaFunction: Função de Notificação

Função orientada a eventos, sem rota HTTP: toda a entrada acontece pela fila **`avaliacoes-criticas`**, que só recebe as avaliações classificadas como críticas pela função de entrada. A função monta e envia o e-mail de urgência aos administradores e **não acessa o banco**.

Estrutura

```
function/NotificarUrgenciaFunction.java     # @QueueTrigger · avaliacoes-criticas
service/NotificacaoService.java             # monta o assunto e o corpo do e-mail
template/EmailTemplates.java                # urgencia(AvaliacaoCriticaDTO), Qute
client/EmailClient.java                     # envia pelo Communication Services
model/dto/notificacao/AvaliacaoCriticaDTO.java  # contrato da mensagem
resources/templates/email/urgencia.html     # HTML do e-mail
```

Fluxo de notificação



Figura 2. Da avaliação crítica ao e-mail dos administradores

Toda notificação passa pelo mesmo ciclo, em três passos:

1. O runtime entrega a mensagem da fila e a função a converte em `AvaliacaoCriticaDTO`.
2. `NotificacaoService.notificarUrgencia(...)` monta o assunto **Avaliação com urgência crítica recebida** e renderiza o template `urgencia.html`.
3. `EmailClient.enviar(...)` envia pelo Communication Services e espera a conclusão. Se o envio não terminar com sucesso, lança `EmailNaoEnviadoException` e a mensagem volta para a fila.

Contrato da mensagem

A mensagem é publicada em Base64 (`QueueMessageEncoding.BASE64`), o formato que o gatilho de fila espera, e leva todos os dados que o enunciado pede para o e-mail de urgência.

Campo	Tipo	Conteúdo
`id`	UUID	Identificador da avaliação gravada
`descricao`	String	Texto escrito pelo estudante
`nota`	Integer	Nota de 0 a 3, já que só as críticas entram na fila
`urgencia`	Urgencia	`CRITICA`
`dataEnvio`	LocalDateTime	Data e hora de envio, no horário de Brasília

Resiliência

- **Retentativa pela própria fila.** Se a função lança exceção, a mensagem volta a ficar visível e é entregue de novo, até 5 vezes (`"maxDequeueCount": 5` no `host.json`). Falhas passageiras do serviço de e-mail se resolvem sem intervenção.
- **Fila poison.** Depois da quinta falha, o runtime move a mensagem para `avaliacoes-criticas-poison`, criada pelo Bicep. Nada se perde e a mensagem pode ser inspecionada e reprocessada.
- **Avaliação preservada.** A gravação no banco foi confirmada antes da publicação, então nenhuma falha de e-mail desfaz a avaliação.

Exemplo: e-mail de urgência

```
De: DoNotReply@<domínio gerenciado pela Azure> · Para: EMAIL_DESTINATARIOS
Assunto: Avaliação com urgência crítica recebida
```

URGÊNCIA CRÍTICA

Nova avaliação precisa de atenção

Descrição	Não consegui acompanhar a aula, o áudio cortou várias vezes
Nota	2 de 10
Urgência	Crítica
Data de envio	12/10/2026 19:45

5. GerarRelatorioFunction: Função de Relatório

Função agendada, sem rota HTTP: o `@TimerTrigger("0 0 11 * * 1")` a dispara toda segunda-feira às 11:00 UTC, que correspondem às **08:00 de Brasília**. A função consulta as avaliações da semana anterior, calcula os indicadores e envia o relatório por e-mail aos administradores. **Só lê**: não grava nem altera dados.

Estrutura

```
function/GerarRelatorioFunction.java        # @TimerTrigger · toda segunda
service/RelatorioService.java               # período, agregações e envio
repository/AvaliacaoRepository.java         # listagem, média e contagens
model/dto/relatorio/                        # RelatorioSemanalDTO e os DTOs por dia e por urgência
template/                                   # EmailTemplates.relatorioSemanal, FormatacaoExtensions
resources/templates/email/relatorioSemanal.html
```

Agendamento e período

- **Periodicidade.** O cron `0 0 11 * * 1` roda em UTC: toda segunda-feira às 11:00.
- **Período.** O `RelatorioService` toma o dia de hoje no horário de Brasília e consulta de 7 dias atrás, à meia-noite, até hoje, à meia-noite, sem incluir hoje. Rodando numa segunda, o relatório cobre exatamente a semana anterior, de segunda a domingo.
- **Disparo manual.** Para a demonstração, a função pode ser executada na hora pela rota administrativa do runtime, `POST /admin/functions/GerarRelatorio`, com a `master-key` do Function App.

Consultas do relatório

Método	O que calcula	Consulta
`listarPorPeriodo`	Avaliações da semana	Panache `list(...)` com `Sort.by("dataEnvio")`
`calcularMediaNotas`	Média das notas	`select avg(a.nota)`
`contarPorDia`	Quantidade por dia	`group by cast(a.dataEnvio as LocalDate)`
`contarPorUrgencia`	Quantidade por urgência	`group by a.urgencia`

> **Intervalo semiaberto.** Todas as consultas usam `dataEnvio >= início and dataEnvio < fim`, então uma avaliação nunca é contada em duas semanas. As agregações são feitas pelo banco, em projeções diretas para os DTOs, e o índice `ix_avaliacao_data_envio` atende o filtro por período das quatro consultas.

Conteúdo do relatório

O e-mail traz a **média das notas** com uma casa decimal (ou `-` quando não houve avaliações), o **total** da semana, a **quantidade por dia**, a **quantidade por urgência** (crítica, média e baixa, nessa ordem) e a descrição, a urgência e a data de envio de **cada avaliação**, como pede o enunciado.

Banco de dados

A solução tem uma única tabela, `avaliacao`, criada pelo Flyway na inicialização da aplicação (`quarkus.flyway.migrate-at-start=true`).

Coluna	Tipo	Regra
`id`	UNIQUEIDENTIFIER	Chave primária (`pk_avaliacao`), gerada como UUID
`descricao`	NVARCHAR(500)	Obrigatória
`nota`	TINYINT	Obrigatória, `CHECK` entre 0 e 10
`urgencia`	VARCHAR(10)	Obrigatória, `CRITICA`, `MEDIA` ou `BAIXA`
`data_envio`	DATETIME2	Obrigatória, horário de Brasília, indexada

Migration	Descrição
`V1__CriarTabelaAvaliacao.sql`	Cria a tabela `avaliacao`, as constraints e o índice `ix_avaliacao_data_envio`.
`V2__PopularBanco.sql`	Só no perfil `test` (`db/test`): cinco avaliações dos últimos dias, uma de cada faixa, para os testes do relatório.

Exemplo: relatório semanal

Relatório enviado na segunda-feira, 12/10/2026, com as avaliações de exemplo.

```
De: DoNotReply@<domínio gerenciado pela Azure> · Para: EMAIL_DESTINATARIOS
Assunto: Relatório semanal de avaliações - 05/10 a 11/10
```

RELATÓRIO SEMANAL

Avaliações de 05/10/2026 a 11/10/2026

Média das notas	Total de avaliações
5,6	**5**

Quantidade de avaliações por dia

07/10/2026	1
09/10/2026	1
10/10/2026	2
11/10/2026	1

Quantidade de avaliações por urgência

Crítica	1
Média	2
Baixa	2

Avaliações recebidas

Descrição	Urgência	Data de envio
Explicação confusa sobre filas	Média	07/10/2026 20:10
O vídeo travou do início ao fim	Crítica	09/10/2026 19:32
Exemplos práticos ajudaram bastante	Baixa	10/10/2026 18:05
Conteúdo bom, mas o áudio falhou em alguns trechos	Média	10/10/2026 21:47
Aula excelente, muito didática	Baixa	11/10/2026 10:15

6. Configuração e Execução

O repositório traz tudo para rodar as funções localmente e criar o ambiente na Azure. Os serviços locais imitam os da Azure (SQL Server e Azurite), então o mesmo código roda nos dois lugares.

Arquivos de configuração

Arquivo	Papel
`docker-compose.yml`	SQL Server 2022 na porta 1433, que cria os bancos `feedback` e `feedback_test`, e Azurite 3.36.0 nas portas 10000 a 10002. Ambos com healthcheck.
`host.json`	Configuração do runtime: rotas sem o prefixo `/api`, 5 tentativas na fila e amostragem do Application Insights sem descartar requisições e exceções.
`local.settings.example.json`	Modelo das variáveis locais do runtime: worker Java, Azurite como storage e perfil `dev`.
`application.properties`	Configuração do Quarkus por perfil (`dev`, `test` e produção).
`.env.example`	Modelo para trocar a senha local do SQL Server.
`infra/`	`main.bicep`, `dev.bicepparam` e os módulos de cada recurso da Azure.
`.run/`	Perfis prontos do IntelliJ para rodar, compilar e testar.

Variáveis de ambiente

Em produção, todas as variáveis são criadas pelo Bicep como configurações do Function App. Nenhuma delas contém senha.

Variável	Descrição
`AzureWebJobsStorage__accountName`	Conta de storage do runtime e da fila, acessada por identidade gerenciada
`APPLICATIONINSIGHTS_CONNECTION_STRING`	Destino dos logs, métricas e traces
`DATABASE_URL`	JDBC do Azure SQL com `authentication=ActiveDirectoryMSI`

Variável	Descrição
`FILA_ENDPOINT`	Endpoint da Storage Queue, acessado por identidade gerenciada
`EMAIL_CONNECTION_STRING`	Referência `@Microsoft.KeyVault(...)` ao segredo do Communication Services
`EMAIL_REMETENTE`	Remetente do domínio gerenciado pela Azure
`EMAIL_DESTINATARIOS`	E-mails dos administradores, separados por vírgula

Segredos. O único segredo da solução é a connection string do Communication Services. O Bicep a grava no Key Vault e o Function App a lê por referência, com a sua identidade gerenciada, então ela nunca aparece no código, no repositório ou nas configurações. No ambiente local, a senha do SQL Server é apenas a padrão do contêiner e pode ser trocada pelo `.env`.

Instruções para execução local

Pré-requisitos: JDK 21 (o Gradle Wrapper baixa o Gradle sozinho), Docker e Docker Compose, e o Azure Functions Core Tools 4 (`npm install -g azure-functions-core-tools@4`) para rodar as funções localmente. Para publicar, também a Azure CLI e uma assinatura Azure.

```
# 1. Serviços locais: SQL Server (1433) e Azurite (10000 a 10002)
docker compose up -d --wait

# 2. Configurações locais do runtime de Functions (perfil dev)
cp local.settings.example.json local.settings.json

# 3. Gera o pacote e sobe as três funções com o Core Tools
./gradlew quarkusRun

# 4. Envia uma avaliação crítica (o e-mail é simulado no log)
curl -X POST http://localhost:8081/avaliacao \
  -H "Content-Type: application/json" \
  -d '{"nota":1,"descricao":"O vídeo não carrega"}'

# Testes de integração (basta o docker compose no ar)
./gradlew test
```

> **Windows.** Utilize `.\gradlew.bat` no lugar de `./gradlew`. O IntelliJ traz as configurações prontas em `.run/`: **QuarkusRun - DEV**, **QuarkusRun - PROD**, **Clean Build - [Without Tests]** e **Testes de Integração**. As variáveis definidas no ambiente têm prioridade sobre o `local.settings.json`, e é assim que os perfis escolhem entre `dev` e `prod`.

Deploy na Azure

A infraestrutura é criada uma vez com o Bicep. Depois, só as funções precisam ser publicadas de novo a cada mudança.

```
# 1. Resource group e infraestrutura completa
az group create --name rg-feedback-dev --location brazilsouth
az deployment group create --resource-group rg-feedback-dev \
  --template-file infra/main.bicep --parameters infra/dev.bicepparam

# 2. Acesso da função ao banco (Query editor, com o administrador Entra ID)
CREATE USER [func-feedback-dev-xxxxxx] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [func-feedback-dev-xxxxxx];
ALTER ROLE db_datawriter ADD MEMBER [func-feedback-dev-xxxxxx];
ALTER ROLE db_ddladmin ADD MEMBER [func-feedback-dev-xxxxxx];

# 3. Pacote e publicação das três funções
FUNCTION_APP_NAME=<function-app> ./gradlew build
cd build/azure-functions/<function-app> && func azure functionapp publish <function-app>

# 4. Chave da função de entrada e teste
az functionapp function keys list --resource-group rg-feedback-dev \
  --name <function-app> --function-name ReceberAvaliacao
curl -X POST "https://<function-app>.azurewebsites.net/avaliacao?code=<chave>" \
  -H "Content-Type: application/json" -d '{"nota":1,"descricao":"O vídeo não carrega"}'
```

Antes do passo 1, preencha `infra/dev.bicepparam` com os e-mails dos administradores e o administrador Entra ID do SQL Server.

O que o Bicep cria

Módulo	Recursos
`function-app`	Plano `FC1` Flex Consumption e Function App Linux com Java 21, identidade gerenciada, HTTPS obrigatório, TLS 1.2 e publicação por FTP e por senha desativadas.
`sql`	Servidor com autenticação somente Entra ID e banco `feedback` serverless, com pausa automática e oferta gratuita.

Módulo	Recursos
`storage`	Conta de storage sem chave compartilhada e com TLS 1.2, contêiner dos pacotes e as filas `avaliacoes-criticas` e `avaliacoes-criticas-poison`.
`email`	Email Communication Service com domínio gerenciado pela Azure e Communication Services, com dados no Brasil.
`keyvault`	Key Vault com autorização RBAC e o segredo `email-connection-string`.
`monitoring`	Log Analytics (30 dias de retenção), Application Insights, grupo de ação por e-mail e os dois alertas.
`rbac`	Papéis de menor privilégio para a identidade da função e acesso de leitura para os membros do grupo.
`main.bicep`	Orquestra os módulos, aplica as tags e cria o orçamento mensal.

Monitoramento

O Application Insights recebe os logs, as métricas e a execução de cada função, ligado a um workspace do Log Analytics. Dois alertas avisam os administradores por e-mail, pelo mesmo grupo de ação:

Alerta	Condição	Severidade
Execuções de função com falha	Alguma execução terminou com erro nos últimos 5 minutos	1
Exceções na aplicação	Mais de 5 exceções em 15 minutos	2

Segurança e governança

- **Sem senhas no código.** A função acessa banco, fila e Key Vault com identidade gerenciada, e o SQL Server aceita apenas autenticação Entra ID.
- **Menor privilégio.** A identidade da função recebe só os papéis de que precisa: *Storage Blob Data Owner* e *Storage Queue Data Contributor* na conta de storage e *Key Vault Secrets User* no cofre. Os membros do grupo entram como *Reader* do resource group.
- **Endpoint protegido.** A única função HTTP exige chave de função, e as respostas de erro 500 não expõem detalhes internos.

- **Transporte protegido.** HTTPS obrigatório, TLS 1.2 mínimo, FTP e publicação por senha desativados, storage sem acesso público e sem chave compartilhada.
- **Governança de custo.** Resource group por ambiente, tags em todos os recursos e orçamento mensal com aviso em 80% do gasto real e em 100% da previsão.

URLs e portas configuradas

Recurso	URL	Porta
Funções (local)	`http://localhost:8081/avaliacao`	`8081`
Função de entrada (Azure)	`https://<function-app>.azurewebsites.net/avaliacao`	`443`
Disparo manual do relatório	`/admin/functions/GerarRelatorio`	`443`
SQL Server (local)	`jdbc:sqlserver://localhost:1433`	`1433`
Azurite (blob / fila / tabela)	`http://127.0.0.1:10000 · :10001 · :10002`	`10000-10002`

Perfis da aplicação

Perfil	Características
`dev`	Banco `feedback` no SQL Server local, fila no Azurite e e-mail simulado no log.
`test`	Banco `feedback_test` com a carga de `V2__PopularBanco.sql`, fila `avaliacoes-criticas-teste` e e-mail simulado, para não misturar dados com o perfil `dev`.
`prod`	Perfil padrão na Azure: banco por `DATABASE_URL` com identidade gerenciada, fila por `FILA_ENDPOINT` e e-mail real pelo Communication Services.

7. Qualidade do Código

O projeto adota as convenções idiomáticas do Quarkus e princípios consolidados de projeto orientado a objetos. A suíte de testes serve de rede de segurança: cada teste sobe o contexto completo do Quarkus e roda contra um SQL Server e um Azurite reais.

11 métodos de teste em 8 classes, todos passando. A função HTTP recebe um dublê de requisição e precisa responder **201**, e os testes de dados rodam contra o banco `feedback_test` populado. Cada teste desfaz as próprias alterações com `@TestTransaction`, então a ordem de execução não altera o resultado.

Funções

Um teste por função, chamando o gatilho como o runtime faria.

`3 testes · JUnit 5`

Services

Um teste por método público: gravação, e-mail de urgência e relatório.

`3 testes · @QuarkusTest`

Dados e regra

Um teste por consulta do repositório e a regra de urgência.

`5 testes · @TestTransaction`

Princípios SOLID e DRY

- **Responsabilidade única.** Cada função tem um gatilho e um motivo para mudar. Dentro dela, a função converte, o service decide, o repository consulta e o client fala com a Azure.
- **Inversão de dependência.** As classes recebem as dependências por construtor, com `@RequiredArgsConstructor` e campos `final`. Os clientes do SDK são produzidos pelo `AzureConfig`, que escolhe entre connection string local e identidade gerenciada sem que os services saibam a diferença.
- **Aberto/fechado.** Uma nova função entra como uma nova classe com o seu gatilho, um novo canal de aviso entra como um novo client e uma nova exceção entra como um novo caso no `FunctionExceptionHandler`, sem alterar o que já existe.
- **DRY.** A regra de urgência existe em um único método, o tratamento de erros em um único handler, a configuração em uma única interface tipada (`FeedbackConfig`) e as chamadas HTTP de teste em uma classe base (`AbstractFunctionTest`).

Modelagem, segurança e integração contínua

- **Imutabilidade e camadas isoladas.** DTOs e mensagens são `record` imutáveis, e a entidade JPA nunca é serializada diretamente: o MapStruct converte entre entidade e DTO.

- **Templates verificados no build.** Com `@CheckedTemplate`, um template Qute que referencia um campo inexistente quebra a compilação em vez de gerar um e-mail errado.
- **Erros padronizados.** Toda resposta de erro segue o mesmo formato, e falhas inesperadas são registradas no log sem expor detalhes ao cliente.
- **Segurança da informação.** Nenhum segredo no código, identidade gerenciada em todos os acessos e o único segredo guardado no Key Vault.
- **Testes de integração reais.** Os testes usam banco e fila próprios (`feedback_test` e `avaliacoes-criticas-teste`), não um banco em memória, e só o envio de e-mail é simulado.
- **Integração contínua.** A cada Pull Request para `main`, o GitHub Actions (`ci.yml`) sobe o SQL Server e o Azurite, configura o Java 21 e executa `./gradlew build`. Uma regressão reprova o PR antes do merge.

8. Repositório do Código

URL

- **Feedback-Serverless**: https://github.com/JavaPosTech/Feedback-Serverless · branch `main` · versão 1.0.0

Organização do repositório

Caminho	Conteúdo
`/src/main/java`	Código-fonte, organizado por camada e por assunto
`/src/main/resources`	`application.properties`, migration do Flyway e templates dos e-mails
`/src/test`	Suíte de testes, carga de dados do perfil de teste e requisições de exemplo
`/infra`	Infraestrutura como código em Bicep
`docker-compose.yml`	SQL Server e Azurite para rodar e testar localmente
`host.json`	Configuração do runtime do Azure Functions
`.github/workflows`	Pipeline de integração contínua a cada Pull Request
`.run`	Perfis de execução do IntelliJ
`README.md`	Documentação de arquitetura, execução, deploy e monitoramento

Funções da Fase 4

Função	Gatilho	Papel
ReceberAvaliacao	HTTP `POST /avaliacao`	Entrada das avaliações. Valida, classifica, grava e enfileira as críticas
NotificarUrgencia	Fila `avaliacoes-criticas`	E-mail de urgência aos administradores. Não acessa o banco
GerarRelatorio	Timer, segunda 08:00 BRT	Relatório semanal com média, quantidade por dia e por urgência. Só lê